

AI Operating System Workshop

Day 3: AI Agents & AI Operating System

AI Agents และ AI Operating System

X-Company

2026

ส่วนที่ 1: สถาปัตยกรรม AI Agent

AI Agent คือระบบอัตโนมัติที่รับรู้สภาพแวดล้อม ใช้เหตุผล และดำเนินการเพื่อบรรลุเป้าหมาย ต่างจาก LLM chatbot ทั่วไป ตรงที่ agent มี memory ถาวร เข้าถึง tools ได้ และวางแผนงานหลายขั้นตอนได้

องค์ประกอบหลัก

องค์ประกอบ	บทบาท	ตัวอย่าง
LLM (สมอง)	Reasoning, planning, เข้าใจภาษา	LLaMA3, Mistral, GPT-4
Tools	โต้ตอบกับโลกภายนอก	ค้นหาเว็บ, คำนวณ, ไฟล์, APIs
Memory	เก็บ context ข้ามเซสชัน	Vector DB, Redis, conversation history
Planning	แบ่งเป้าหมายเป็นขั้นตอน	ReAct, Chain-of-Thought, Tree-of-Thought

วงจร Agent: รับรู้ → คิด → กระทำ

```
# วงจร Agent (simplified)
while not goal_achieved:

    # 1. รับรู้: สังเกตสถานะปัจจุบัน

    observation = get_observation() # input ผู้ใช้, ผล tool ฯลฯ

    # 2. คิด: วิเคราะห์การกระทำถัดไป
    thought = llm.think(observation, memory, available_tools)

    action = parse_action(thought) # เช่น {'tool': 'search', 'args': {'q': 'Python docs'}}

    # 3. กระทำ: ดำเนินการ tool ที่เลือก
    result = execute_tool(action)
    memory.append({'observation': observation, 'action': action, 'result': result})

    goal_achieved = check_goal(result)
```

ส่วนที่ 2: Tool-Use และ Function Calling

การตั้งค่า Function Calling ของ Ollama

```
import ollama

# กำหนด tools
tools = [
    {
        'type': 'function',
        'function': {
```

```

        'name': 'calculator',
        'description': 'คำนวณทางคณิตศาสตร์',
        'parameters': {
            'type': 'object',
            'properties': {
                'expression': {'type': 'string',
                                'description': 'นิพจน์คณิตศาสตร์ที่ต้องการ
คำนวณ'}
            },
            'required': ['expression']
        }
    }
]

response = ollama.chat(
    model='llama3.2',

    messages=[{'role': 'user', 'content': '15 * 24 + 7 เท่ากับเท่าไร?'}],
    tools=tools
)

```

ตัวอย่าง: Calculator + Search Tools

```

import math, requests

def calculator(expression: str) -> str:
    try:
        result = eval(expression, {'__builtins__': {}}, vars(math))
        return str(result)
    except Exception as e:
        return f'Error: {e}'

def web_search(query: str) -> str:
    resp = requests.get(f'https://api.duckduckgo.com/?q={query}&format=json')

    return resp.json().get('AbstractText', 'ไม่พบผลลัพธ์')

TOOL_MAP = {'calculator': calculator, 'web_search': web_search}

def handle_tool_call(tool_call):
    fn = TOOL_MAP[tool_call['function']]['name']
    args = tool_call['function']['arguments']
    return fn(**args)

```

ส่วนที่ 3: RAG Agent

RAG Agent รวมพลังการดึงความรู้ของ RAG เข้ากับความสามารถในการวางแผนและใช้ tools ของ agent โดย agent จะตัดสินใจว่า เมื่อไหร่จะค้นฐานความรู้ ค้นหาอะไร และรวมผลลัพธ์อย่างไร

```

import ollama
from qdrant_client import QdrantClient

client = QdrantClient('localhost', port=6333)

```

```
def rag_tool(query: str) -> str:
    q_vec = ollama.embeddings(model='bge-m3', prompt=query)['embedding']
    results = client.search('workshop_docs', query_vector=q_vec, limit=5)
    return '\n'.join([r.payload['text'] for r in results])

TOOL_MAP = {'rag_tool': rag_tool}

# วงจร Agent กับ RAG
messages = [{'role': 'user', 'content': user_question}]
while True:
    response = ollama.chat(model='llama3.2', messages=messages, tools=tools)
    if response['message'].get('tool_calls'):
        for tc in response['message']['tool_calls']:
            result = handle_tool_call(tc)
            messages.append({'role': 'tool', 'content': result})
    else:
        print(response['message']['content'])
        break
```

ส่วนที่ 4: ระบบ Multi-Agent

ภาพรวม Patterns

Pattern	คำอธิบาย	กรณีใช้งาน	ตัวอย่าง
Sequential	A → B → C (pipeline)	สายการประมวลผล	ดึง → สรุป → แปล
Parallel	A + B + C (concurrent)	งานอิสระหลายอย่าง	ค้นหาหลายแหล่งพร้อมกัน
Routing	Router → A หรือ B หรือ C	Agents เฉพาะทาง	จัดประเภทคำถาม → ส่งไปผู้เชี่ยวชาญ
Orchestrator	Manager → สร้าง workers	งานระยะยาวซับซ้อน	Project manager agent มอบหมายงาน

Orchestrator Pattern

```
class OrchestratorAgent:
    def __init__(self, workers: dict):
        self.workers = workers

    def run(self, task: str) -> str:

        # 1. วางแผนงาน
        plan = self.llm_plan(task)

        # plan = [{'agent': 'research', 'input': 'เทรนด์ AI 2026'},
        #         {'agent': 'write', 'input': 'ร่างรายงานจากงานวิจัย'}]

        context = {}
        for step in plan:
            agent_name = step['agent']
            result = self.workers[agent_name].run(step['input'])
            context[f'{agent_name}_result'] = result
```

```
return context.get('write_result', 'งานเสร็จสิ้น')
```

ส่วนที่ 5: AI Operating System

AI Operating System (AI OS) คือแพลตฟอร์มที่ประสาน AI agents, skills, แหล่งข้อมูล และการทำงานอัตโนมัติตามเวลา — คล้ายกับ OS ทั่วไปที่ประสาน processes, I/O และ scheduling

องค์ประกอบหลัก

องค์ประกอบ	หน้าที่	Analogy
Agents	ดำเนินงานด้วย LLM + tools	OS processes
Skills	โมดูล capability ที่นำกลับมาใช้ได้	OS libraries / drivers
Memory	เก็บ state ถาวร	File system + RAM
Heartbeat	ตรวจสอบพื้นหลังเป็นระยะ	OS cron / system daemon
Cron	รันงานตามเวลาที่กำหนด	OS cron scheduler
Message Bus	การสื่อสาร agent-to-agent	IPC / message queue

คำอธิบายสถาปัตยกรรม

สถาปัตยกรรม AI OS ประกอบด้วย 3 ชั้น:

- Infrastructure Layer: LLM servers (Ollama), Vector DB (Qdrant), Graph DB (Neo4j), Object storage
- Platform Layer: Agent runtime, Skill registry, Memory manager, Event bus, Scheduler
- Application Layer: Domain agents (HR, Finance, Sales), User interfaces (Telegram, Web), APIs

ส่วนที่ 6: การประเมินด้วย RAGAS

RAGAS (Retrieval-Augmented Generation Assessment) ให้ metrics อัตโนมัติในการประเมินคุณภาพระบบ RAG โดยไม่ต้องใช้ label จากมนุษย์

4 Metrics หลัก

Metric	วัดอะไร	สูตร	เป้าหมาย
Faithfulness	คำตอบอ้างอิง context ครบไหม?	Claims ที่มีหลักฐาน / Claims ทั้งหมด	> 0.8
Answer Relevancy	คำตอบตอบคำถามไหม?	Cosine sim(answer, question)	> 0.8
Context Precision	chunks ที่ดึงมาเกี่ยวข้อง	Chunks เกี่ยวข้อง /	> 0.7

	ไหม?	ทั้งหมดที่ตั้ง	
Context Recall	ดึงข้อเท็จจริงสำคัญ ครบไหม?	ข้อเท็จจริงที่ตั้ง / ทั้งหมด	> 0.7

รัน RAGAS

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision,
context_recall
from datasets import Dataset

data = {
    'question': ['นโยบายการคืนเงินเป็นอย่างไร?'],
    'answer': ['การคืนเงินใช้เวลา 7 วันทำการ'],
    'contexts': [['ลูกค้าสามารถขอคืนเงินได้ภายใน 30 วัน...']],
    'ground_truth': ['การคืนเงินใช้เวลา 7 วันทำการ']
}
dataset = Dataset.from_dict(data)

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall]
)
print(result)
```

แนวทาง LLM-as-Judge

ใช้ LLM ประเมินคุณภาพคำตอบเมื่อไม่มี ground truth:

```
judge_prompt = '''
ให้คะแนน RAG response นี้ 1-5 ในแต่ละเกณฑ์:

- Faithfulness: ทุก claim มีหลักฐานจาก context?
- Relevance: ตอบคำถามได้?
- Completeness: ครอบคลุมทุกด้านของคำถาม?

คำถาม: {question}
Context: {context}

คำตอบ: {answer}

ส่งกลับ JSON: {"faithfulness": int, "relevance": int, "completeness": int}
'''
```

A/B Testing Methodology

- กำหนด baseline metric (คะแนน RAGAS ของระบบปัจจุบัน)
- สร้าง variant B (เปลี่ยน chunking, โมเดล, prompt ฯลฯ)
- รัน evaluation บน test set เดียวกันทั้ง A และ B
- ความมีนัยสำคัญทางสถิติ: ใช้ t-test หรือ bootstrap (n > 30 ตัวอย่าง)
- เกณฑ์ตัดสินใจ: ปรับปรุง > 5% บน metric หลัก

ส่วนที่ 7: โปรเจกต์สุดท้าย (Capstone)

สร้าง Enterprise AI Assistant

เป้าหมาย: สร้าง AI assistant พร้อม production สำหรับองค์กร ที่รวมแนวคิดทั้งหมดจากการอบรม

ข้อกำหนด

- Document ingestion: รองรับ PDF, DOCX พร้อมประมวลผลภาษาไทย
- Hybrid search: Dense + BM25 กับ RRF fusion
- Knowledge graph: ดึง entity และเก็บใน Neo4j
- Multi-tool agent: อย่างน้อย 3 tools (RAG, ค้นหา, คำนวณ)
- Evaluation: คะแนน RAGAS บน test set 50 คำถาม
- UI: Telegram bot หรือ web interface ง่าย

เกณฑ์การประเมิน

เกณฑ์	น้ำหนัก	คะแนนขั้นต่ำ
Faithfulness (RAGAS)	25%	0.75
Answer Relevancy (RAGAS)	25%	0.75
Context Precision (RAGAS)	20%	0.70
คุณภาพโค้ดและเอกสาร	15%	ผ่าน
Demo และการนำเสนอ	15%	ผ่าน

ส่วนที่ 8: แบบฝึกหัดและเอกสารอ้างอิง

แบบฝึกหัดที่ 1: Agent พร้อม Tools

งาน: สร้าง agent ที่มี 3 tools: web search, calculator, และ RAG จัดการ tool chaining

- Hint: ใช้ while loop จนกว่าจะไม่มี tool_calls ใน response

แบบฝึกหัดที่ 2: RAGAS Evaluation

งาน: สร้างชุด Q&A 20 คู่จากระบบ RAG ของคุณ ประเมินด้วย RAGAS หาจุดอ่อน

- Hint: มุ่งเน้น Faithfulness — ระบบ RAG ส่วนใหญ่ล้มเหลวตรงนี้ก่อน

แบบฝึกหัดที่ 3: Multi-Agent Orchestration

งาน: สร้างระบบ 2 agents: Research agent + Summary agent จัดการแบบ sequential

- Hint: ส่ง research_result เป็น context ให้ summary agent

เอกสารอ้างอิง

- Ollama Tools API: <https://ollama.ai/blog/tool-support>
- RAGAS: <https://docs.ragas.io>
- LangGraph (Multi-Agent): <https://langchain-ai.github.io/langgraph>
- OpenClaw (AI OS): <https://openclaw.ai>
- ReAct Paper: <https://arxiv.org/abs/2210.03629>